RidgeRestore

AI-Powered Fingerprint Restoration using conditional GAN, our WE Capstone Project

Shruthi Simran Plakshi

March 12, 2026

Table of Contents

1. Introduction
2. Dataset and Preprocessing
3. Model Architecture
4. Training and Performance
5. Results and Evaluation
6. Full-Stack Application
7. Challenges Faced
8. Learnings
9. Future Scope

Motivation

The Problem

- Fingerprints are the most used biometric identifier
- Real-world prints are often partial or degraded
- Low-quality prints fail in AFIS matching
- Hand-crafted filters have limited effectiveness

Impact

Degraded prints directly hinder:

- Criminal investigations
- Border security
- Biometric authentication

Project Overview

What is RidgeRestore?

A full-stack AI web app that restores degraded fingerprint images using a **Conditional GAN**, wrapped in a forensic-grade web interface.

Approach

- U-Net Generator reconstructs ridge patterns
- PatchGAN Discriminator enforces texture realism
- Model served via REST API to a React frontend

Deliverables

- Trained cGAN model
- Flask REST API
- React + TypeScript web app
- **82% MSE improvement**

Team Contributions

Member	Domain	Key Contributions
Shruthi	Backend & Model	U-Net Generator, PatchGAN Discriminator, Flask API, model export & MSE evaluation
Simran	Model & Integration	Noise injection pipeline, dataset curation, hyperparameter tuning, API integration & testing
Plakshi	Frontend & Integration	React + TypeScript frontend, TIFF upload, comparison view, UI design

Background & Related Work

Traditional Methods

- **Gabor Filters** — ridge enhancement, fails on heavy degradation
- **Histogram Equalisation** — improves contrast, introduces artifacts
- **STFT** — local frequency analysis, computationally expensive

Deep Learning Approaches

- **DnCNN** — good for Gaussian noise, poor ridge reconstruction
- **Autoencoders** — blurry outputs due to pixel-averaging loss
- **Pix2Pix cGAN** — best sharpness via adversarial loss ✓

Our Choice: cGAN with U-Net Generator + PatchGAN Discriminator.

Dataset Overview

Statistics

- Total: **320 images**
- Format: Grayscale `.tif`
- Train / Test: 80% / 20%
- Setup: Supervised paired learning

Tensor Config

Shape	$(320, 128, 128, 1)$
Size	128×128
Range	$[0, 1]$
Batch	8

Preprocessing Pipeline

1. **Border Cropping** — remove scanner frame artifacts
2. **Resize to** 128×128 — standardise spatial dimensions
3. **Normalise to** $[0, 1]$ — min-max pixel scaling
4. **Tensor Conversion** — shape $(320, 128, 128, 1)$
5. **Gaussian Noise Injection** — construct paired $\langle$noisy, clean$\rangle$ training data

Data Augmentation

Techniques

- Horizontal and vertical flips
- Random rotations ($\pm 15^\circ$)
- Brightness and contrast jitter
- Gaussian blur variations

Effect

Stage	Samples
Original	256
Augmented	$\sim$1200+

Note

All augmentations applied **identically** to both noisy and clean images to preserve paired structure.

Conditional GAN Framework

Generator: U-Net

- **6.17M** parameters
- Encoder-decoder + skip connections
- Input/Output: 128×128
- LeakyReLU $\rightarrow$ ReLU $\rightarrow$ Tanh

Discriminator: PatchGAN

- **663K** parameters
- Patch output: $16 \times 16 \times 1$
- Classifies patches (real vs. fake)
- Input: [noisy | generated]

Why Conditional?

Both G and D are conditioned on the noisy input, guiding restoration toward the correct target rather than any plausible output.

Model Architecture Diagram

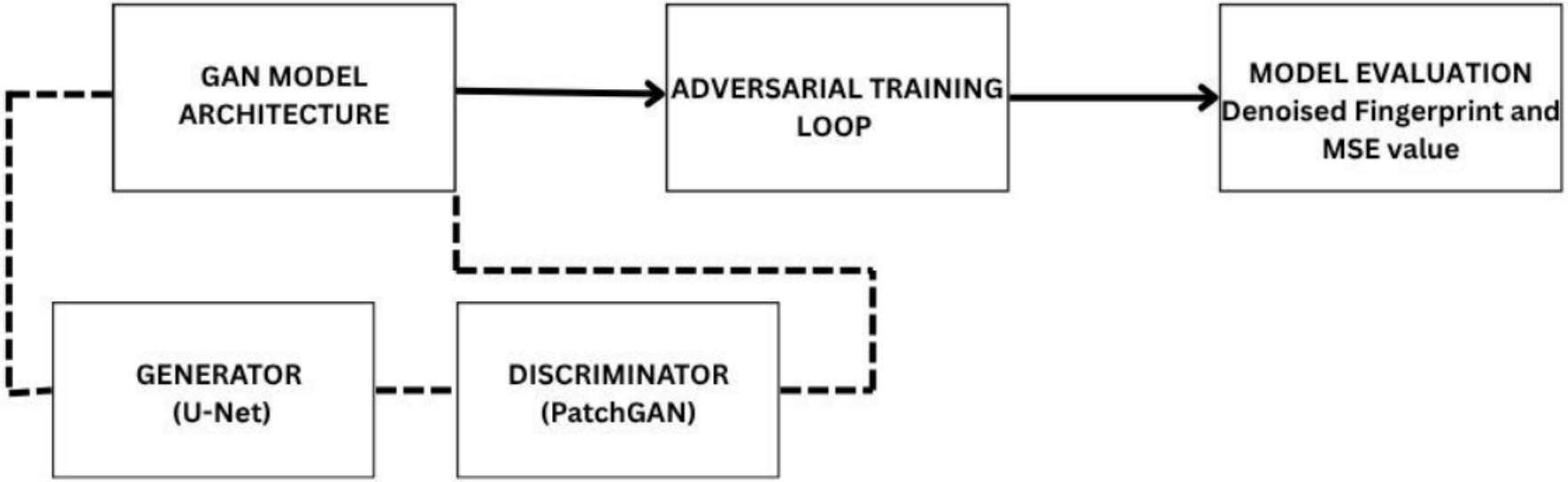

Figure: U-Net Generator + PatchGAN Discriminator

U-Net Generator — Details

Encoder

- 8 conv blocks, stride-2 downsampling
- Batch Norm + LeakyReLU ($\alpha = 0.2$)
- Channels: $64 \to 128 \to 256 \to 512$

Decoder

- Transposed conv upsampling
- Dropout ($p = 0.5$, first 3 blocks)
- Channels: $512 \to 256 \to 128 \to 64$

Skip Connections

Encoder feature maps concatenated to decoder layers, preserving fine ridge detail lost at the bottleneck.

Bottleneck

Densest feature representation. No Batch Norm at this layer.

PatchGAN Discriminator — Details

Architecture

- 5 convolutional layers, no FC layers
- Stride-2 conv for downsampling
- Batch Norm + LeakyReLU ($\alpha = 0.2$)
- Channels: $64 \to 128 \to 256 \to 512$
- Final: 1×1 conv $\to$ sigmoid

Input

- Concatenation of noisy + generated image
- Input shape: $128 \times 128 \times 2$

Output

Produces a 16×16 map of real/fake scores. Each score covers a 70×70 receptive field — local judgement, not a single global verdict.

Why PatchGAN?

Patch-level discrimination penalises local texture inconsistencies, enforcing ridge-scale sharpness that a global discriminator would miss.

Loss Functions

Generator Loss

$$\mathcal{L}_G = \mathcal{L}_{\text{BCE}}(D(x, G(x)), 1) + \lambda\, \mathcal{L}_{L1}(G(x), y), \quad \lambda = 100$$

Discriminator Loss

$$\mathcal{L}_D = \mathcal{L}_{\text{BCE}}(D(x, y), 1) + \mathcal{L}_{\text{BCE}}(D(x, G(x)), 0)$$

Optimiser

- Adam, lr $= 2 \times 10^{-4}$
- $\beta_1 = 0.5, \ \beta_2 = 0.999$

Why L1 + BCE?

BCE alone causes blur. L1 enforces pixel accuracy. Together: sharp, correct restorations.

Training Configuration

Parameter	Value
Epochs	100
Batch Size	8
Optimiser	Adam
Learning Rate	2×10^{-4}
β_1	0.5
λ	100
Framework	TensorFlow / Keras

Strategy

- G and D updated once per batch
- Alternating updates prevent dominance
- Checkpoint saved at lowest val loss

Training Performance

Loss Summary

Metric	Value
Initial Gen Loss	13.81
Final Gen Loss	6.28
Reduction	$\approx 54\%$
Discriminator	Balanced

Observations

- Rapid drop: epochs 0–20
- Gradual gain: epochs 20–60
- Stable plateau: epoch 60+
- No mode collapse

Learning Curve

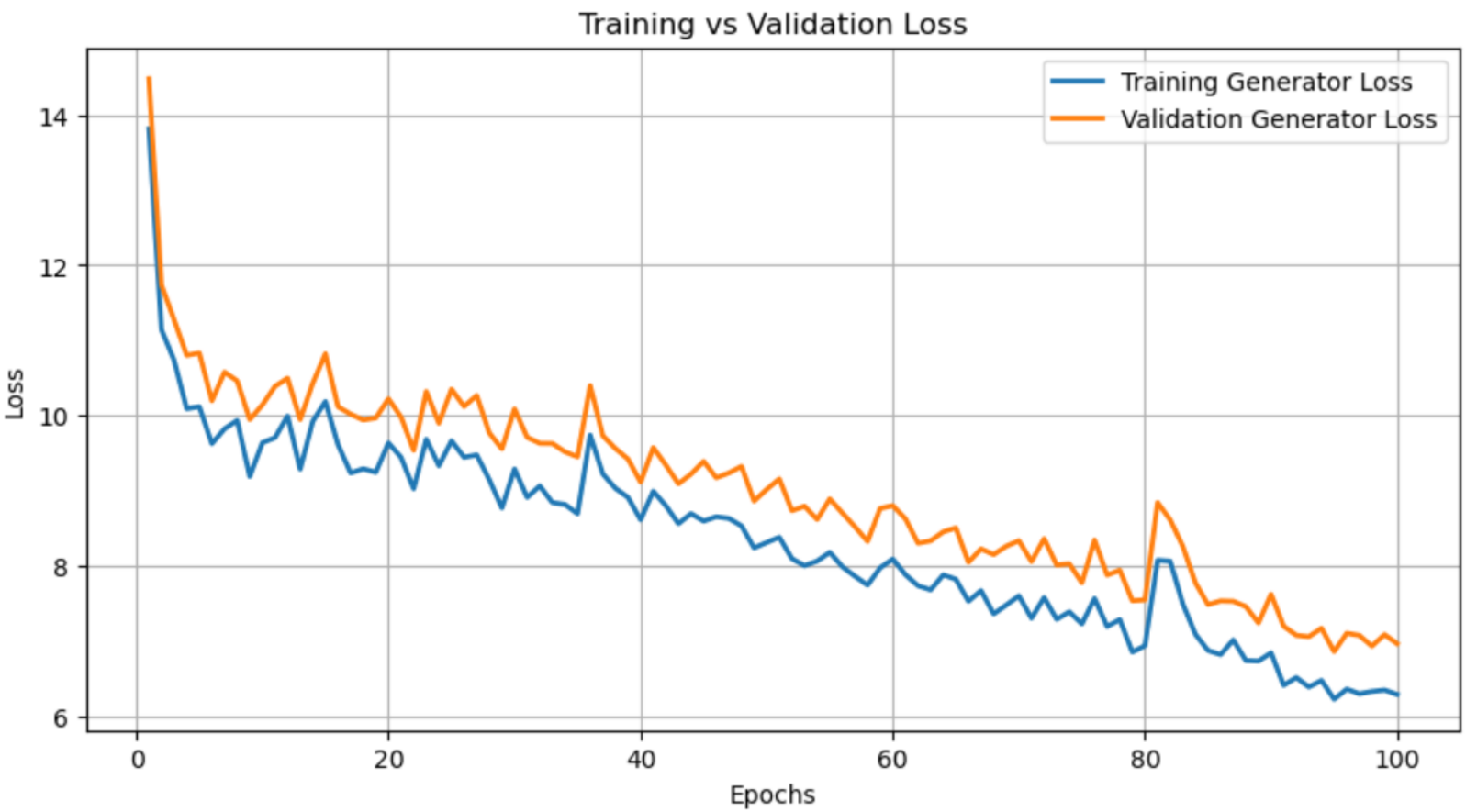

Figure: Training vs. Validation Generator Loss over 100 Epochs

- Smooth descent confirms stable GAN training
- Train and validation curves closely aligned — no overfitting

Reconstruction Results

- Clear ridge enhancement across all test samples
- Gaussian noise removed without loss of ridge topology
- Minutiae points (bifurcations, endings) preserved
- No false ridges introduced

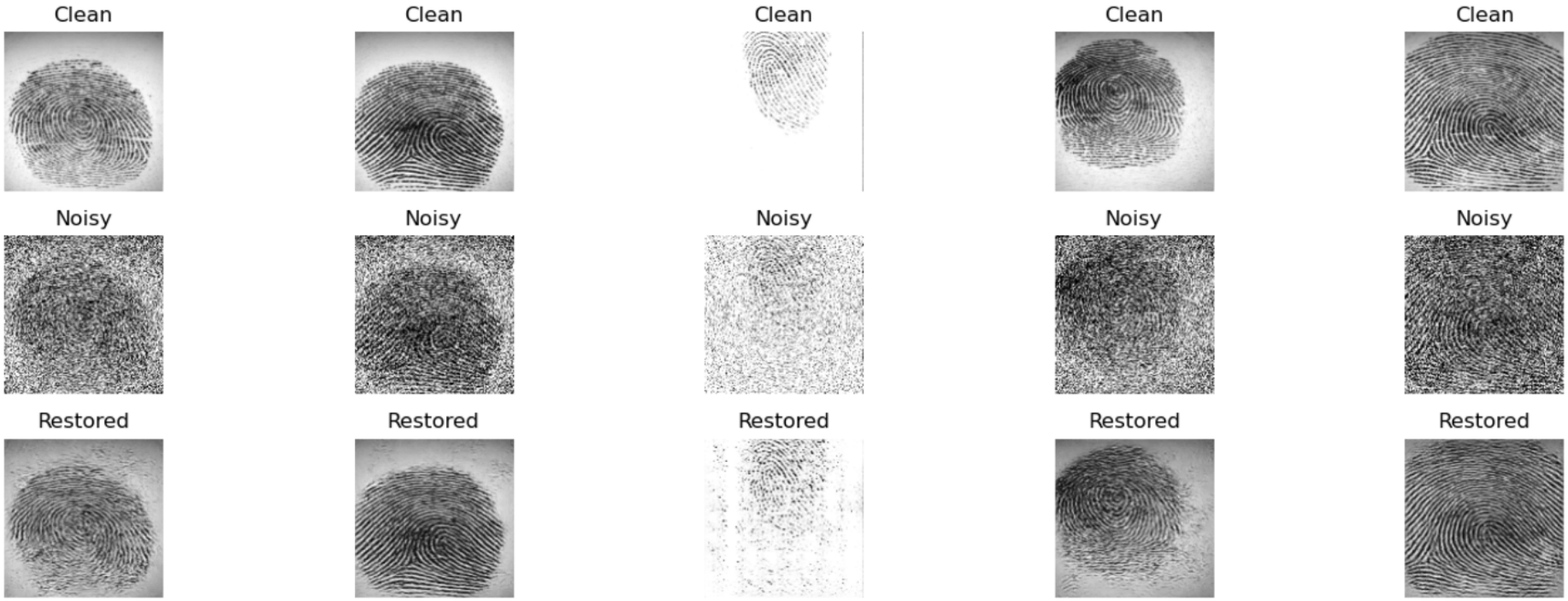

Figure: Clean Reference — Noisy Input — Restored Output

Quantitative Evaluation

MSE Comparison

Comparison	MSE
Clean vs. Noisy	0.0643
Clean vs. Restored	**0.0111**
Improvement	**82%**

Key Result

Restored image is **6**× **closer** to the clean reference than the noisy input.

```
✅ Mean Squared Error on 5 test samples: 0.011190070580965711
10/10 ━━━━━━━━━━━━━━━━━━━━ 8s 755ms/step
✅ Mean Squared Error on full dataset: 0.011169361500242298
✅ Mean Squared Error between clean and noisy images: 0.06429837024100854
```

Figure: MSE Calculation

Technology Stack

Frontend

- React 18 + TypeScript
- Vite, Tailwind CSS, shadcn/ui
- TIFF upload, side-by-side compare, PNG download

Backend & AI

- Python / Flask REST API
- TensorFlow / Keras
- NumPy + Pillow
- Flask-CORS

API

`POST /restore` — accepts `.tif` file, returns base64-encoded original + restored PNG as JSON.

Application Workflow

1. **Upload** — user drops a `.tif` file onto the React interface
2. **Validation** — client checks file type and size
3. **Processing** — Flask decodes TIFF via Pillow, runs U-Net inference
4. **Response** — base64-encoded original + restored images returned as JSON
5. **Display & Download** — side-by-side comparison; user saves restored PNG

Processing Time: 2–5 seconds per fingerprint on standard CPU.

Challenges Faced

1. Limited Dataset (320 images)

Risk of overfitting. $\rightarrow$ Resolved with augmentation and synthetic noise injection.

2. GAN Training Instability

Early loss oscillation and mode collapse risk. $\rightarrow$ Tuned $\beta_1 = 0.5$, lr$= 2 \times 10^{-4}$, $\lambda = 100$.

3. TIFF in Browser

Native browsers cannot render `.tif` files. $\rightarrow$ Server-side TIFF-to-PNG via Pillow before response.

4. Model File Size

200 MB `.h5` exceeded standard Git limits. $\rightarrow$ Used Git LFS for model versioning.

Learnings — Deep Learning

- **cGANs vs. Autoencoders:** Adversarial loss eliminates blurriness that pixel-averaging in autoencoders produces
- **L1 Loss ($\lambda = 100$):** Without it, the generator hallucinates incorrect ridges; L1 anchors outputs to the target
- **PatchGAN:** Local discrimination preserves high-frequency ridge texture better than global binary discrimination
- **Skip Connections:** Essential — ridge detail lost at the bottleneck is recovered only through skip paths
- **Hyperparameter Sensitivity:** Small changes to lr and β_1 caused dramatic shifts in GAN convergence

Learnings — Engineering & Integration

Data Engineering

- Noise injection is effective for paired data construction
- Preprocessing consistency matters as much as model design
- Augmentation helps but cannot replace real data diversity

Full-Stack

- Load ML models at startup, not per request
- Serialise images as base64 for clean REST transport
- CORS and env vars are critical in production
- TypeScript guards complex async image flows well

Future Scope

Model

- Larger, more diverse training datasets
- Transformer-GAN with attention
- SSIM and PSNR evaluation metrics
- Higher resolution (256×256)
- GPU / CUDA acceleration

Application

- **Authorised sign-in via ID verification**
- Batch processing
- Format conversion (JPG/PNG $\rightarrow$ TIFF)
- Enhancement parameter controls
- Mobile app (iOS / Android)
- AFIS system integration

Thank You

RidgeRestore — Restoring Identity, One Ridge at a Time

Shruthi	**Simran**	**Plakshi**
Backend & Model	Model & Integration	Frontend & Integration

Capstone Project · 2025–26